



KIẾN TRÚC VÀ THIẾT KẾ PHẦN MỀM

Chương 7: Các mẫu thiết kế cấu trúc

ThS. Nguyễn Văn Hữu Hoàng

I NỘI DUNG

Nội dung của chương này tập trung vào các mẫu thiết kế:

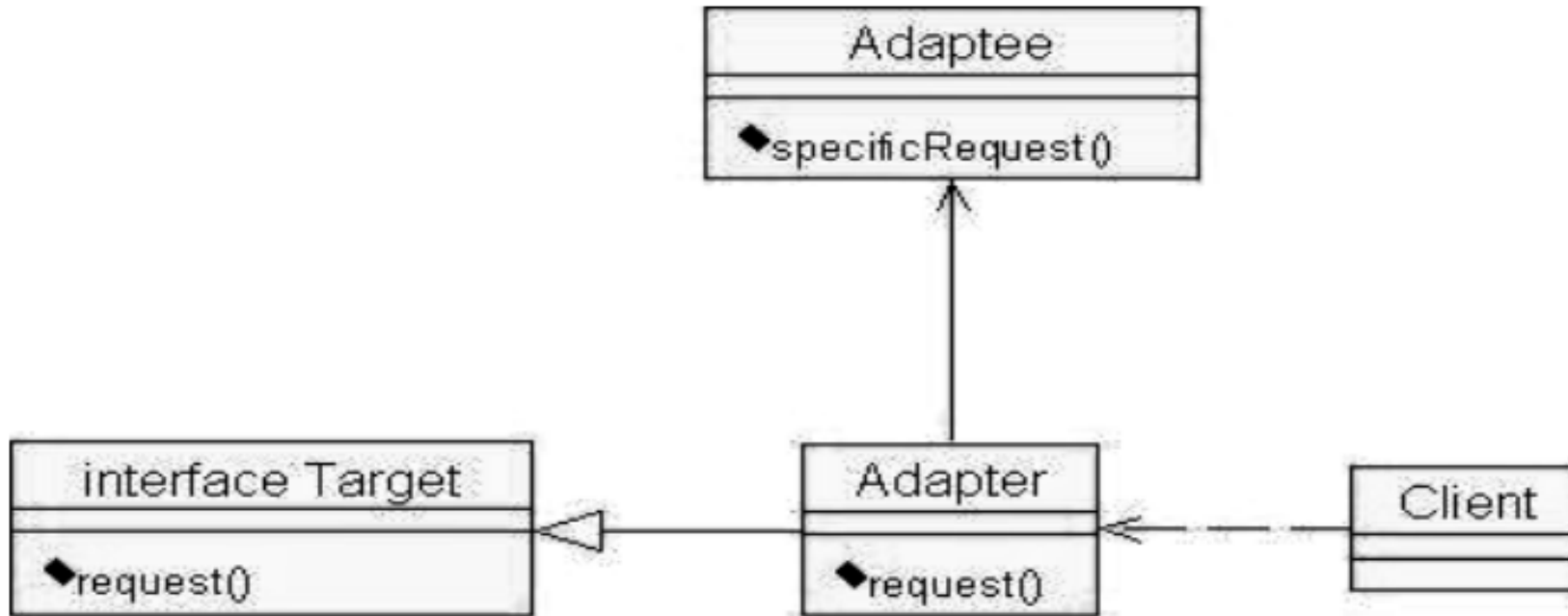
- Mẫu Adapter
- Mẫu Bridge
- Mẫu Composite
- Mẫu Decorator
- Mẫu Facade
- Mẫu Flyweight
- Mẫu Proxy

| 7.1 MẪU THIẾT KẾ ADAPTER

7.1.1 Đặt vấn đề

- Khi một lớp công cụ được thiết kế nhằm sử dụng lại nhưng không thể dùng được do giao diện của nó không thích hợp với ứng dụng được yêu cầu.
 - Ta muốn sử dụng một lớp đã tồn tại, tuy nhiên giao diện của nó không phù hợp với giao diện của một lớp mà ta yêu cầu. Vì vậy ta muốn tạo ra một lớp có khả năng dùng lại bằng cách kết hợp lớp đó với các lớp không liên quan hoặc không được dự đoán trước và có giao diện không tương thích với nhau.
- Mẫu **adapter** là giải pháp cho vấn đề này.
- **Adapter** là mẫu thiết kế dùng để biến đổi giao diện của một lớp thành một giao diện khác mà client yêu cầu. Nó giúp cho các lớp không thể hoạt động với nhau bằng cách biến đổi giao diện sao cho chúng có thể tương thích với nhau.

| 7.1.2 CẤU TRÚC MẪU ADAPTER



- **Target** là một **Interface** xác định các chức năng, yêu cầu mà **Client** cần sử dụng.
- **Adaptee** là lớp chứa các chức năng mà **Target** cần sử dụng nhằm tạo ra các chức năng mà **Target** cần cung cấp cho **Client**.
- **Adapter** cài đặt phương thức từ **Target** và sử dụng đối tượng lớp **Adaptee**, **Adapter** có nhiệm vụ gắn kết **Adaptee** vào **Target** để có được chức năng mà **Client** mong muốn.

| 7.1.3 TÌNH HUỐNG ÁP DỤNG MẪU ADAPTER

- Khi ta muốn sử dụng một lớp có sẵn nhưng cổng giao tiếp của nó không tương thích với yêu cầu hiện tại.
- Muốn tạo một lớp tái sử dụng có thể hoạt động được với những lớp khác không liên hệ gì với nó và không nhất thiết phải tương thích giao diện với nhau.

| 7.1.4 VÍ DỤ MẪU ADAPTER

- Một hệ thống quản lý Phone cần thực hiện một số chức năng nào đó, trong đó có một phương thức trả về số điện thoại tương ứng với một chuỗi ký tự đầu vào.
- Giả sử trước đó ta đã có một lớp có chức năng chuyển các ký tự số từ một chuỗi và ta muốn sử dụng chức năng đó vào hệ thống lấy số điện thoại. Mã nguồn như sau:

```
//Giao tiếp PhoneTarget
public interface PhoneTarget{
    //lấy số điện thoại trong một chuỗi
    public String getPhoneNumber(String message);
}

public GetNumberAdaptee{
    public String getNumber(String str){
        //lấy ra dạng số trong một chuỗi
        <...get number>
    } ...
}

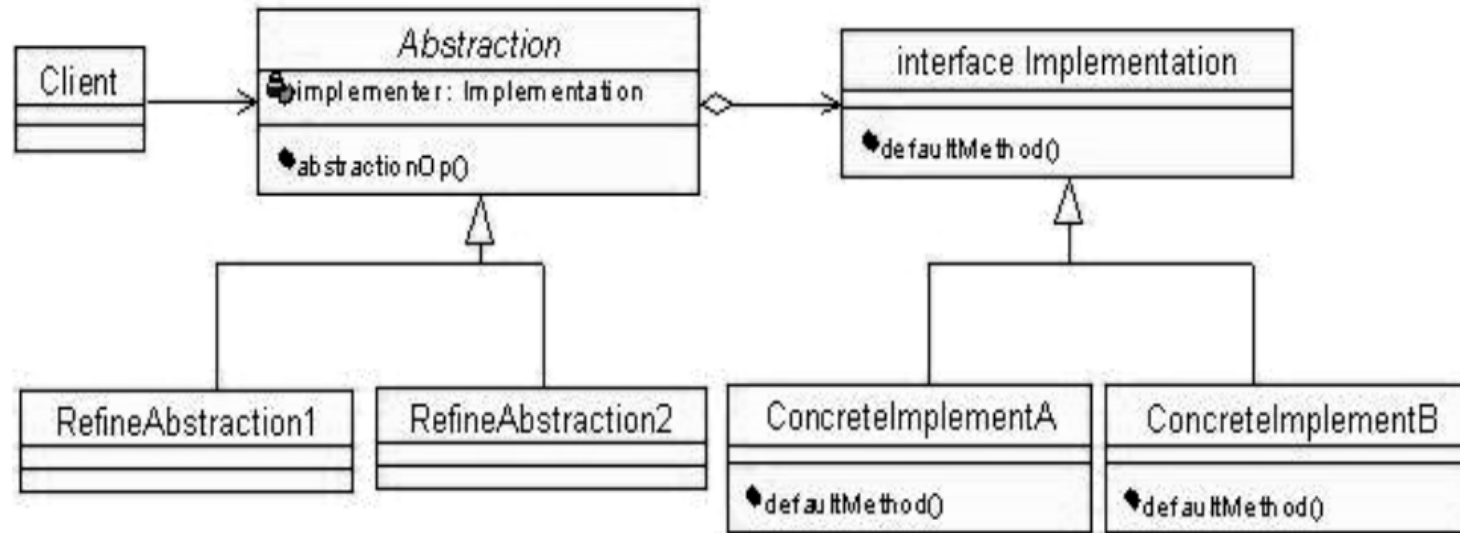
public Adapter implements PhoneTarget{
    public String getPhoneNumber(String message){
        GetNumberAdaptee obj = new GetNumberAdaptee();
        String str = obj.getNumber(message);
        return "84"+str;
    }
}
```

| 7.2 MẪU THIẾT KẾ BRIDGE

7.2.1 Đặt vấn đề

- Khi một lớp trừu tượng cần bổ sung thêm một vài thành phần thì cách thông thường là sử dụng kế thừa. Nghĩa là định nghĩa một giao tiếp cho lớp trừu tượng đó và các lớp con cụ thể thực hiện nó theo các cách khác nhau. Nhưng cách tiếp cận như vậy thường là không đủ mềm dẻo. Do tính kế thừa ràng buộc thành phần cần bổ sung thêm, nên sẽ khó thay đổi, mở rộng, và sử dụng lại các lớp trừu tượng.
- Trong trường hợp như vậy, sử dụng một mẫu **Bridge** là thích hợp nhất.
- **Bridge** là mẫu thiết kế sử dụng **Interface** để tách riêng cài đặt phương thức của một lớp trừu tượng để hai đối tượng đó có thể biến đổi độc lập với nhau.

| 7.2.2 CẤU TRÚC MẪU BRIDGE



- **Abstraction**: là một lớp trừu tượng khai báo các chức năng và cấu trúc cơ bản. Lớp này có một thuộc tính là một thể hiện của giao tiếp **interface Implementation**, thể hiện sẽ thực hiện các chức năng **abstractionOp()** của lớp **Abstraction**.
- **Interface Implementation**: là giao tiếp thực thi của lớp các chức năng nào đó của **Abstraction**.
- **RefineAbstractionX**: là định nghĩa các chức năng mới hoặc các chức năng đã có trong **Abstraction**.
- **ConcreteImplementY**: là các lớp định nghĩa tường minh các thực thi trong lớp giao tiếp **Implementation**

| 7.2.3 TÌNH HUỐNG ÁP DỤNG BRIDGE

Mẫu **Bridge** thường được áp dụng khi:

- Ta muốn tránh một ràng buộc cố định giữa một abstraction và một thành phần bổ sung thêm của nó.
- Thay đổi trong thành phần được bổ sung thêm của một abstraction mà không ảnh hưởng đối với các client. Nghĩa là ta muốn làm ẩn đi hoàn toàn các thành phần bổ sung thêm của một abstraction khỏi các client.
- Khi có một sự phát triển rất nhanh các lớp, hệ thống phân cấp lớp chỉ ra là cần phải tách một đối tượng thành hai phần. Khi đó ta muốn tạo ra sự mềm dẻo giữa hai thành phần trừu tượng và thực thi của một thành phần, và tránh đi mối quan hệ tĩnh giữa chúng.

I TÌNH HUỐNG ÁP DỤNG BRIDGE

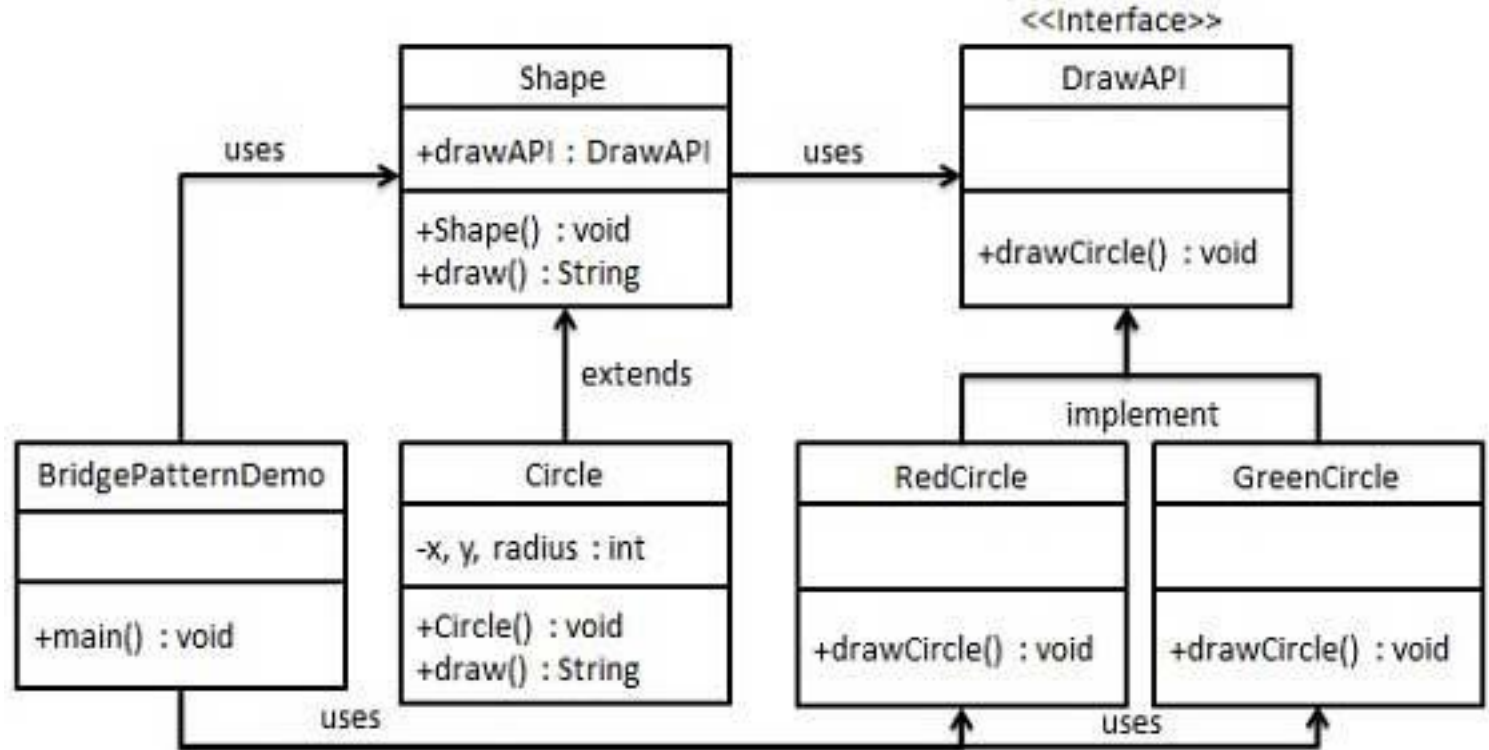
Bridge có một cấu trúc tương tự như một đối tượng của **Adapter**, nhưng **Bridge** có mục đích khác. Nó chia giao diện từ các phần cài đặt của nó ra riêng rẽ để từ đó có thể linh hoạt hơn và độc lập nhau trong khi **Adapter** đồng nghĩa với việc thay đổi giao diện của một đối tượng đã tồn tại.

| 7.2.4 VÍ DỤ MẪU BRIDGE

- Ví dụ sau đây sẽ chỉ ra việc sử dụng mẫu Bridge để vẽ hình tròn có màu khác nhau bằng cách dùng cùng một phương thức của lớp trừu tượng nhưng thể hiện cài đặt khác nhau qua lớp giao tiếp **Bridge**.

I VÍ DỤ MẪU BRIDGE

- Interface **DrawAPI** đóng vai trò như như triển khai cầu nối và các lớp cụ thể. **RedCircle**, **GreenCircle** cài đặt interface tiếp này.
- Shape** là một lớp trừu tượng và sẽ sử dụng đối tượng của **DrawAPI**.
- Lớp **BridgePatternDemo** sẽ sử dụng lớp **Shape** để vẽ hình tròn với màu khác nhau.



| VÍ DỤ MẪU BRIDGE

Bước 1: Tạo bridge interface

DrawAPI.java

```
public interface DrawAPI {  
    public void drawCircle(int radius, int x, int y);  
}
```

Bước 2: Tạo các lớp triển khai DrawAPI

RedCircle.java

```
public class RedCircle implements DrawAPI {  
    @Override  
    public void drawCircle(int radius, int x, int y) {  
        System.out.println("Draw Circle[color:red,radius:"+radius+",x:"+x+",y+"]");  
    }  
}
```

GreenCircle.java

```
public class GreenCircle implements DrawAPI {  
    @Override  
    public void drawCircle(int radius, int x, int y) {  
        System.out.println("Draw Circle[color:green,radius:"+radius+",x:"+x+",y+"]");  
    }  
}
```

Bước 3: Tạo lớp trừu tượng Shape sử dụng DrawAPI

Shape.java

```
public abstract class Shape {  
    protected DrawAPI drawAPI;  
  
    protected Shape(DrawAPI drawAPI){  
        this.drawAPI = drawAPI;  
    }  
    public abstract void draw();  
}
```

| VÍ DỤ MẪU BRIDGE

Bước 4: Tạo lớp cụ thể kế thừa từ Shape

Circle.java

```
public class Circle extends Shape {
    private int x, y, radius;

    public Circle(int x, int y, int radius, DrawAPI drawAPI) {
        super(drawAPI);
        this.x = x;
        this.y = y;
        this.radius = radius;
    }

    public void draw() {
        drawAPI.drawCircle(radius,x,y);
    }
}
```

Bước 6: Theo dõi kết quả.

```
Draw Circle[color: red, radius: 10, x: 100, 100]
Draw Circle[color: green, radius: 10, x: 100, 100]
```

Bước 5: Sử dụng Shape và DrawAPI để vẽ các hình tròn khác nhau.

BridgePatternDemo.java

```
public class BridgePatternDemo {
    public static void main(String[] args) {
        Shape redCircle = new Circle(100,100, 10, new RedCircle());
        Shape greenCircle = new Circle(100,100, 10, new GreenCircle());

        redCircle.draw();
        greenCircle.draw();
    }
}
```

| 7.3 MẪU THIẾT KẾ COMPOSITE

7.3.1 Đặt vấn đề

- Các ứng dụng đồ họa như bộ công cụ soạn thảo hình vẽ và các hệ thống lưu giữ biểu đồ cho phép người sử dụng xây dựng các biểu đồ phức tạp khác xa với các thành phần nguyên thủy và đơn giản ban đầu. Người sử dụng có thể nhóm một số các thành phần để tạo ra các thành phần khác lớn hơn, và các thành phần lớn hơn này lại có thể được nhóm lại để tạo ra các thành phần lớn hơn nữa.
- Nhưng có một vấn đề với cách tiếp cận này là mã sử dụng các lớp đó phải tác động lên các đối tượng nguyên thủy và các đối tượng bao hàm các thành phần nguyên thủy ấy là khác nhau ngay cả khi thời gian người sử dụng tác động lên chúng là như nhau. Sự phân biệt các đối tượng này làm cho ứng dụng trở nên phức tạp hơn.
- Mẫu **Composite** xem xét việc sử dụng các thành phần đệ quy để làm cho các client không có sự phân biệt trên.

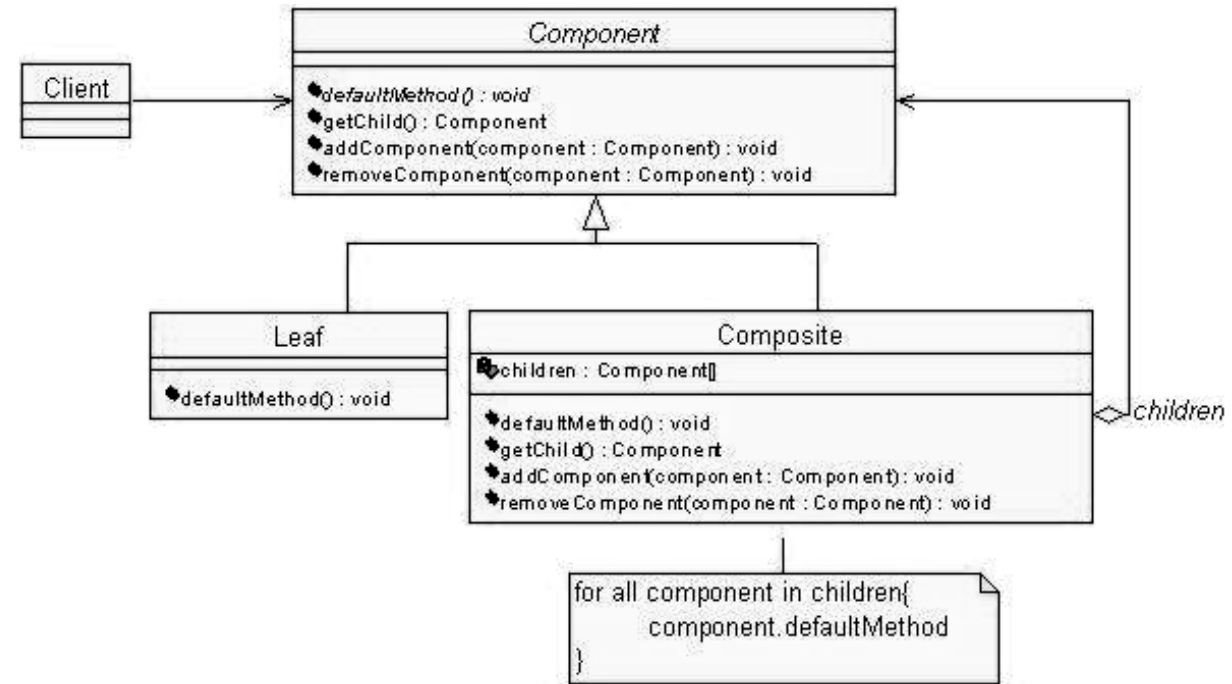
| 7.3 MẪU THIẾT KẾ COMPOSITE

7.3.1 Đặt vấn đề

- Giải pháp của mẫu **Composite** là xây dựng một lớp trừu tượng để biểu diễn cả hai thành phần nguyên thủy và các lớp chứa chúng. Lớp này cũng xác định các thao tác truy nhập và quản lý các đối tượng con của nó.
- Như vậy, **Composite** là mẫu thiết kế dùng để tạo ra các đối tượng trong các cấu trúc cây để biểu diễn hệ thống phân lớp: bộ phận – toàn bộ.
- **Composite** cho phép các client tác động đến từng đối tượng và các thành phần của đối tượng một cách thống nhất.

| 7.3.2 CẤU TRÚC MẪU COMPOSITE

- **Component**: là một giao tiếp interface định nghĩa các phương thức cho tất cả các phần của cấu trúc cây.
- **Composite**: là lớp được định nghĩa bởi các thành phần mà nó chứa. **Composite** chứa một nhóm các **Component**, vì vậy nó có các phương thức để thêm vào hoặc loại bỏ các thể hiện của **Component**. Những phương thức được định nghĩa trong **Component** được thực thi để thực hiện các hành vi đặc tả cho lớp **Composite**. Lớp **Composite** được gọi là lớp nhánh.
- **Leaf**: là lớp thực thi từ giao tiếp **Component**. Sự khác nhau giữa lớp **Leaf** và **Composite** là lớp **Leaf** không chứa các tham chiếu đến các **Component** khác, lớp **Leaf** đại diện cho mức thấp nhất của cấu trúc cây.



| 7.3.3 TÌNH HUỐNG SỬ DỤNG MẪU COMPOSITE

- Khi có một mô hình thành phần với cấu trúc nhánh-lá, toàn bộ-bộ phận, ...hay khi cấu trúc có thể có vài mức phức tạp và động.
- Mẫu thường dùng làm thành phần liên kết đến đối tượng cha là chuỗi trách nhiệm (Chain of Responsibility) sẽ được trình bày sau.
- Mẫu **Decorator**, **Iterator**, **Visitor** cũng thường được sử dụng với **Composite**. Các mẫu này sẽ được xem xét trong phần tiếp theo.

| 7.3.3 VÍ DỤ MẪU COMPOSITE

- Ta hãy xem xét ví dụ dự án **Project**, một **Project** là một hợp nhiều tác vụ **Task** (**Leaf**), ta cần tính tổng thời gian của dự án thông qua thời gian của tất cả các tác vụ.
- Mã nguồn như sau:

```
public interface TaskItem{
    public double getTime();
}

public class Project implements TaskItem{
    private String name;
    private ArrayList subtask = new ArrayList();
    public Project(){ }
    public Project(String newName){
        name = newName;
    }
    public String getName(){ return name; }
    public void setName(String newName){ name = newName; }
    public ArrayList getSubtasks(){ return subtask; }

    public double getTime(){
        double totalTime = 0;
        Iterator items = subtask.iterator();
        while(items.hasNext()){
            TaskItem item = (TaskItem)items.next();
            totalTime += item.getTime();
        }
        return totalTime;
    }
    public void addTaskItem(TaskItem element){
        if (!subtask.contains(element)){
            subtask.add(element);
        }
    }
    public void removeTaskItem(TaskItem element){
        subtask.remove(element);
    }
}
```

| VÍ DỤ MẪU COMPOSITE

```
public class Task implements TaskItem{
    private String name;
    private double time;
    public Task(){ }
    public Task(String newName, double newTimeRequired){
        name = newName;
        time = newTimeRequired;
    }
    public String getName(){ return name; }
    public double getTime(){
        return time;
    }
    public void setName(String newName){ name = newName; }
    public void setTime(double newTimeRequired){
        time = newTimeRequired;
    }
}
```

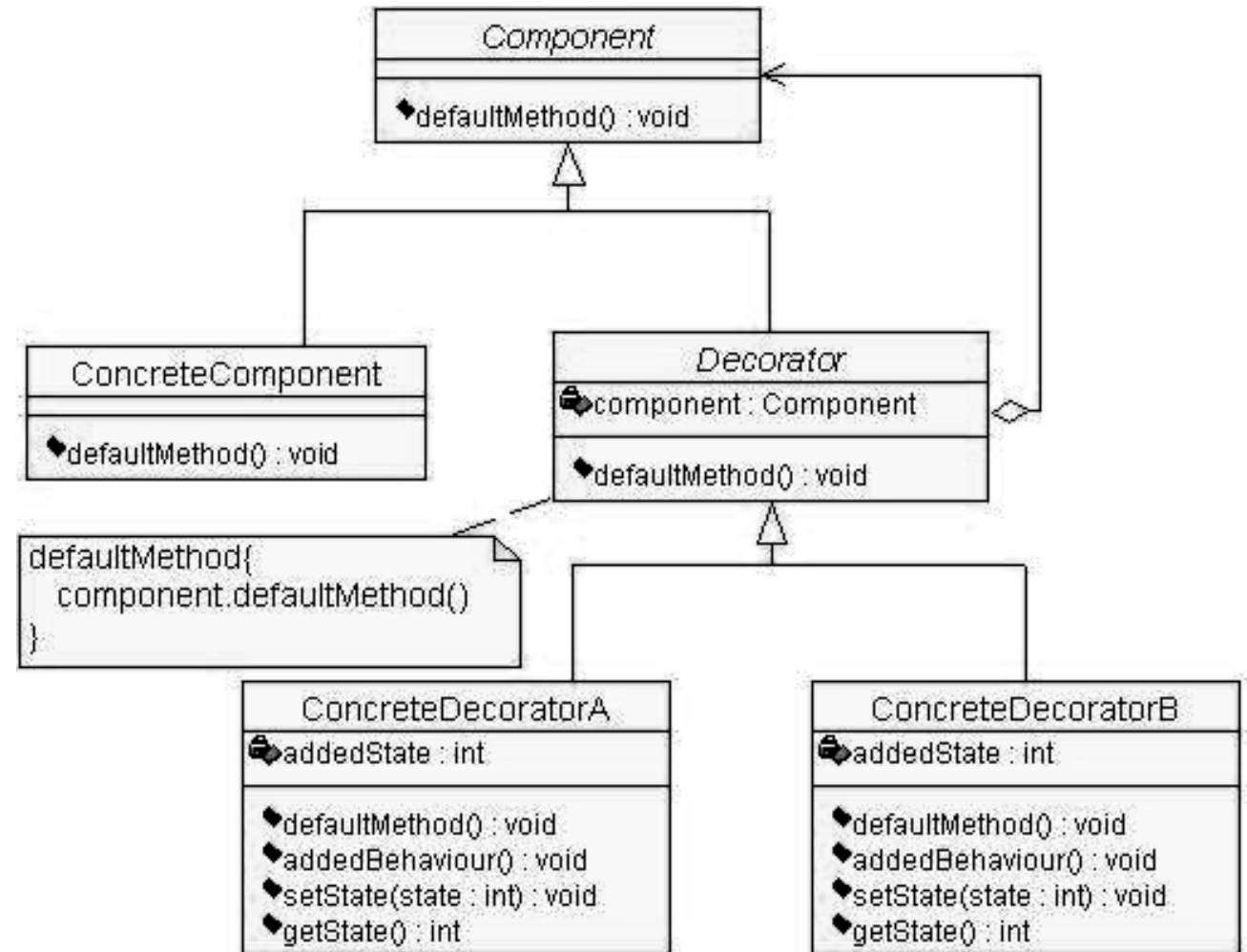
| 7.4 MẪU THIẾT KẾ DECORATOR

7.4.1 Đặt vấn đề

- Mẫu **decorator** được sử dụng để mở rộng chức năng của đối tượng mà không phải thay đổi mã nguồn ban đầu hay sử dụng kế thừa. Điều này có thể thực hiện được bằng cách tạo một đối tượng bao phủ quanh đối tượng đã có với việc gắn các chức năng bổ sung cho các đối tượng đó (gán động).

| 7.4.2 CẤU TRÚC MẪU DECORATOR

- **Component**: là một interface chứa các phương thức ảo (ở đây là **defaultMethod**).
- **ConcreteComponent**: là một lớp kế thừa từ **Component**, cài đặt các phương thức cụ thể (**defaultMethod** được cài đặt tường minh)
- **Decorator**: là một lớp ảo kế thừa từ **Component** đồng thời cũng chứa một thể hiện của **Component**, phương thức **defaultMethod** trong **Decorator** sẽ được thực hiện thông qua thể hiện này.
- **ConcreteDecoratorX**: là các lớp kế thừa từ **Decorator**, khai báo tường minh các phương thức, đặc biệt trong các lớp này khai báo tường minh các “trách nhiệm” cần thêm vào khi trong thời gian chạy.

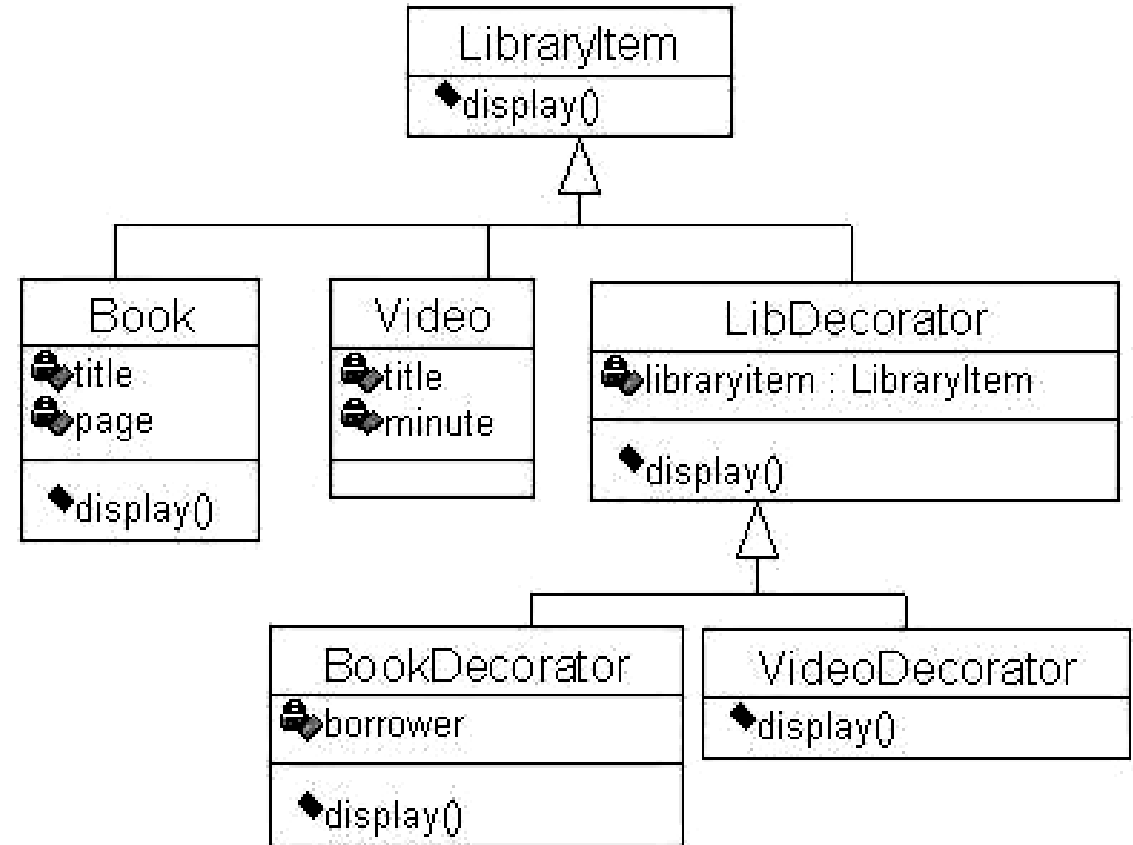


| 7.4.3 TÌNH HUỐNG ÁP DỤNG MẪU DECORATOR

- Khi chúng ta muốn thay đổi động mà không ảnh hưởng đến người dùng và không phụ thuộc vào giới hạn các lớp con; muốn một thành phần có thể thêm vào hoặc loại bỏ khi hệ thống đang chạy. Hoặc khi một số đặc tính phụ thuộc mà bạn muốn áp dụng một cách động và muốn kết hợp chúng vào trong một thành phần.
- Mẫu **Decorator** khác với **Adapter**, **Decorator** chỉ thay đổi nhiệm vụ của đối tượng mà không thay đổi giao diện của nó như **Adapter**. **Adapter** mang đến cho đối tượng một giao diện mới hoàn toàn. **Decorator** cũng có thể coi như một **Composite** bị thoái hoá với duy nhất một thành phần.
- **Decorator** kết hợp thêm phần nhiệm vụ vào đối tượng và cho phép chúng ta thay đổi bề ngoài của một đối tượng. Trong khi đó, mẫu **Strategy** cho phép chúng ta thay đổi bên trong của đối tượng. Chúng là hai cách có thể thay phiên nhau để ta thay đổi một đối tượng.

| 7.4.3 VÍ DỤ MẪU DECORATOR

- Giả sử trong thư viện có các tài liệu: sách, video... Các loại tài liệu này có các thuộc tính khác nhau, phương thức hiển thị của giao tiếp **LibraryItem** sẽ khác nhau.
- Giả sử, ngoài các thông tin về các thuộc tính trên, đôi khi ta muốn hiển thị thông tin về độc giả đã mượn một cuốn sách nào đó, hoặc muốn xem một đoạn video. **Decorator** sẽ hỗ trợ xây dựng đối tượng thực hiện nhiệm vụ này.
- Giao tiếp interface **LibraryItem** định nghĩa phương thức **display()** cho tất cả các tài liệu của thư viện và các lớp tài liệu tương ứng. Các decorator cho **Book** và **Video** kế thừa từ decorator của thư viện **LibDecorator**.



VÍ DỤ MẪU DECORATOR

```
//Định nghĩa giao tiếp
public interface LibraryItem{
    public void display(); //đây là defaultMethod
}

//Định nghĩa các lớp tài liệu
public class Book implements LibraryItem{
    private String title;
    private int page;
    public Book(String s, int p){
        title = s;
        page = p;
    }
    public void display(){
        System.out.println("Title: " + title);
        System.out.println("Page number: " + page);
    }
}

public class Video implements LibraryItem{
    private String title;
    private int minutes;
    public Video(String s, int m){
        title = s;
        minutes = m;
    }
    public void display(){
        System.out.println("Title: " + title);
        System.out.println("Time: " + minutes);
    }
}
```

```
//Lớp trừu tượng Decorator thu viện
public abstract class LibDecorator implements LibraryItem{
    private LibraryItem libraryitem;
    public LibDecorator(LibraryItem li){
        libraryitem = li;
    }
    public void display(){
        libraryitem.display();
    }
}

//Các lớp Decorator cho mỗi tài liệu thư viện cần bổ sung
//trách nhiệm ở thời điểm chạy
public class BookDecorator extends LibDecorator{
    private String borrower;
    public BookDecorator(LibraryItem li, String b){
        super(li);
        borrower = b;
    }
    public void display(){
        super.display();
        System.out.println("Borrower: " + borrower);
    }
}

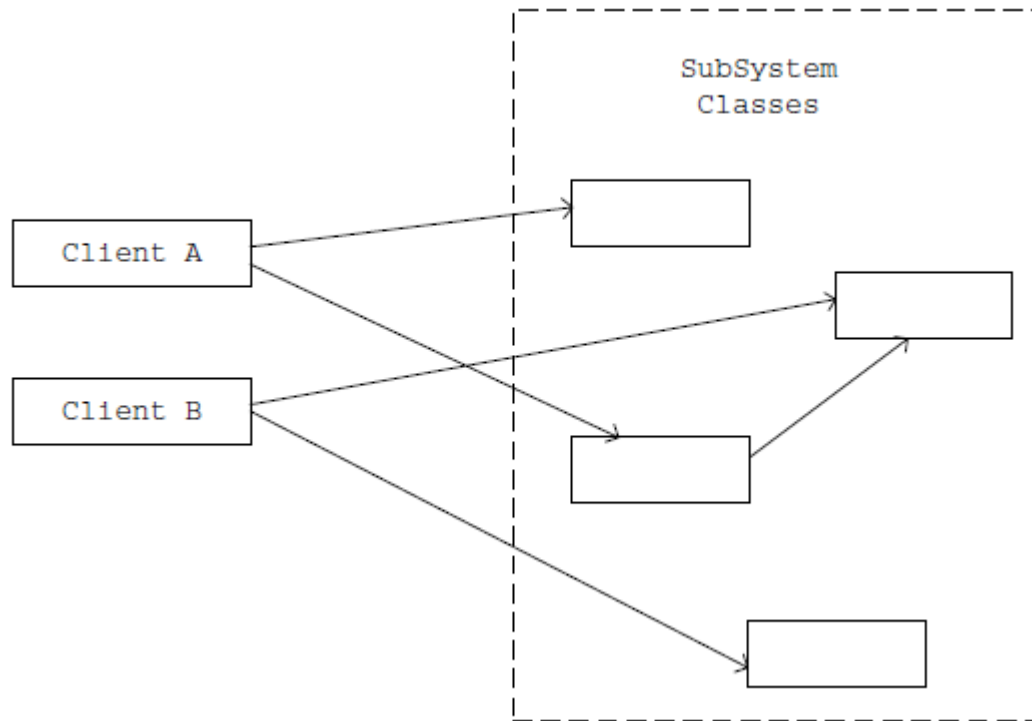
public class VideoDecorator extends LibDecorator{
    public VideoDecorator(LibraryItem li){
        super(li);
    }
    public void display(){
        super.display();
        play(); //phương thức play video
    }
}
```

| 7.5 MẪU THIẾT KẾ FACADE

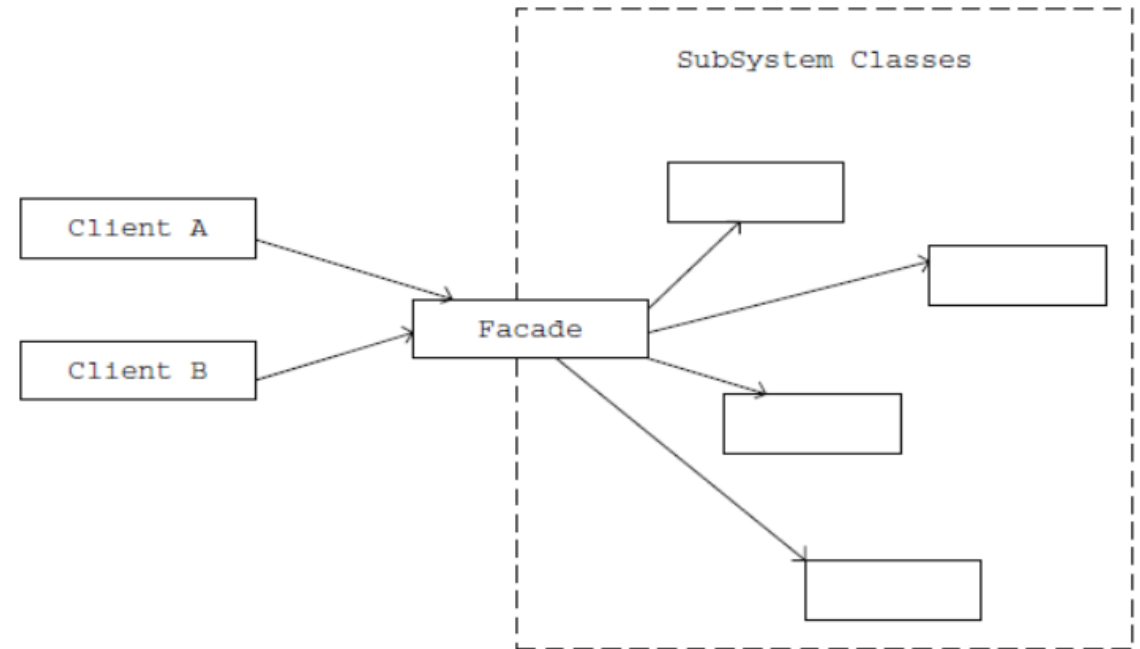
7.5.1 Đặt vấn đề

- Thông thường để giảm độ phức tạp, ta sẽ phân rã một hệ thống thành các hệ thống con trong đó mỗi hệ thống con bao gồm các lớp liên kết nhau để cung cấp các chức năng nào đó. Ví dụ, lớp **Account**, địa chỉ **Address** và thẻ tín dụng **CreditCard** hoạt động cùng nhau để cung cấp các chức năng trực tuyến cho khách hàng.
- Thông thường thiết kế một hệ thống là phải tuân theo nguyên lý sao cho tối thiểu hóa được sự giao tiếp và phụ thuộc giữa các hệ thống con. Một cách để đạt được mục tiêu này là đưa ra đối tượng **facade** nhằm cung cấp một giao diện để client dễ dàng giao tiếp với hệ thống con.
- Nghĩa là khi đó các client sẽ tương tác với đối tượng **facade** thay vì tương tác với các hệ thống con và vai trò tương tác với các hệ thống con do **facade** đảm nhiệm.
- Mẫu **facade** cung cấp một giao diện thống nhất cho một tập các giao diện trong một hệ thống con và như vậy sẽ làm cho các hệ thống con được sử dụng dễ dàng hơn.

| MẪU THIẾT KẾ FACADE

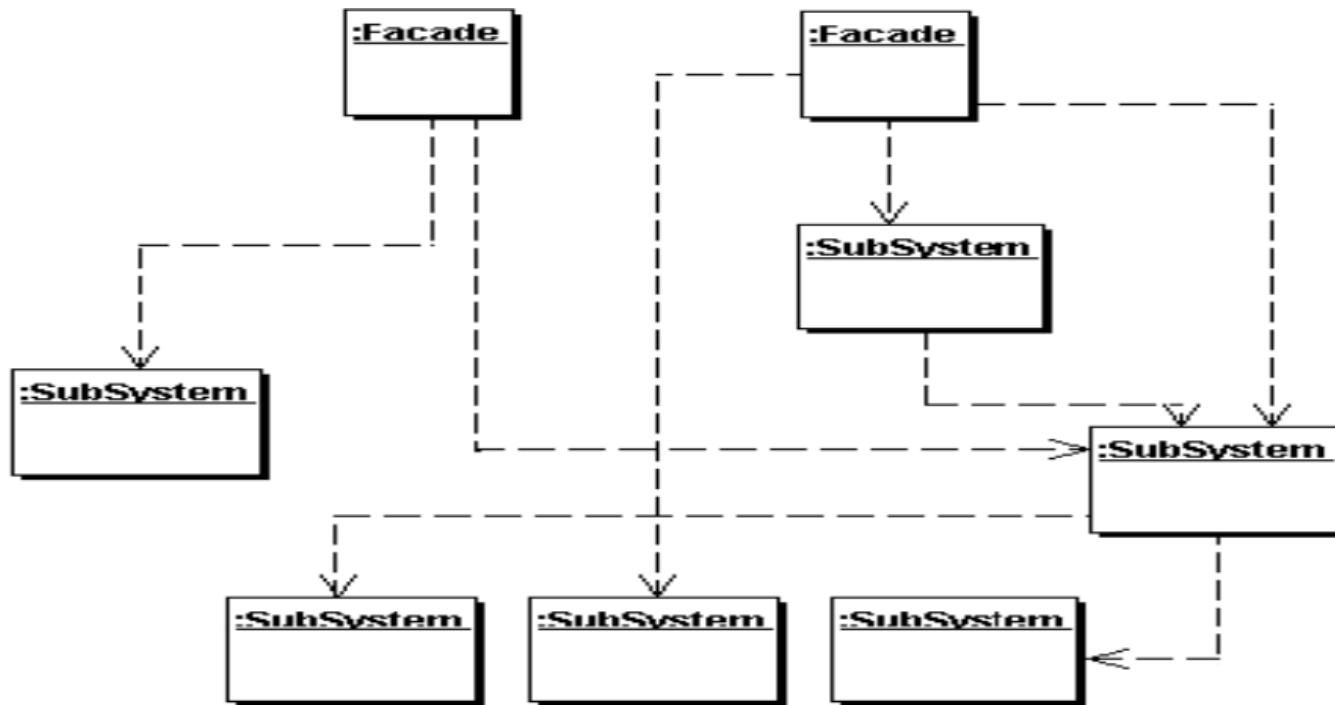


Tương tác với hệ thống con không có facade

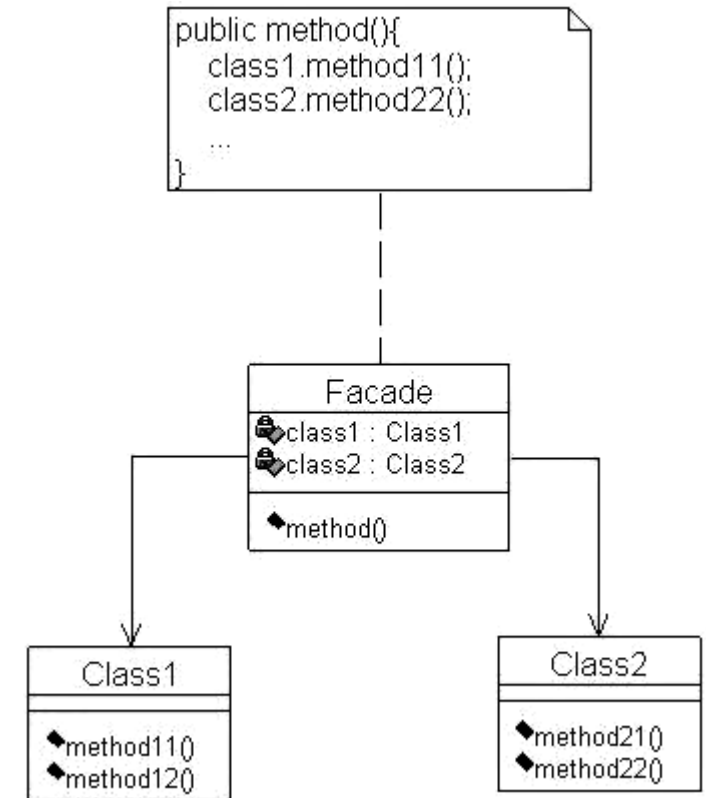


Tương tác với hệ thống con qua facade

| 7.5.2 CẤU TRÚC MẪU THIẾT KẾ FACADE



Kiến trúc mẫu Facade



Ví dụ cụ thể với Facade

Trong đó:

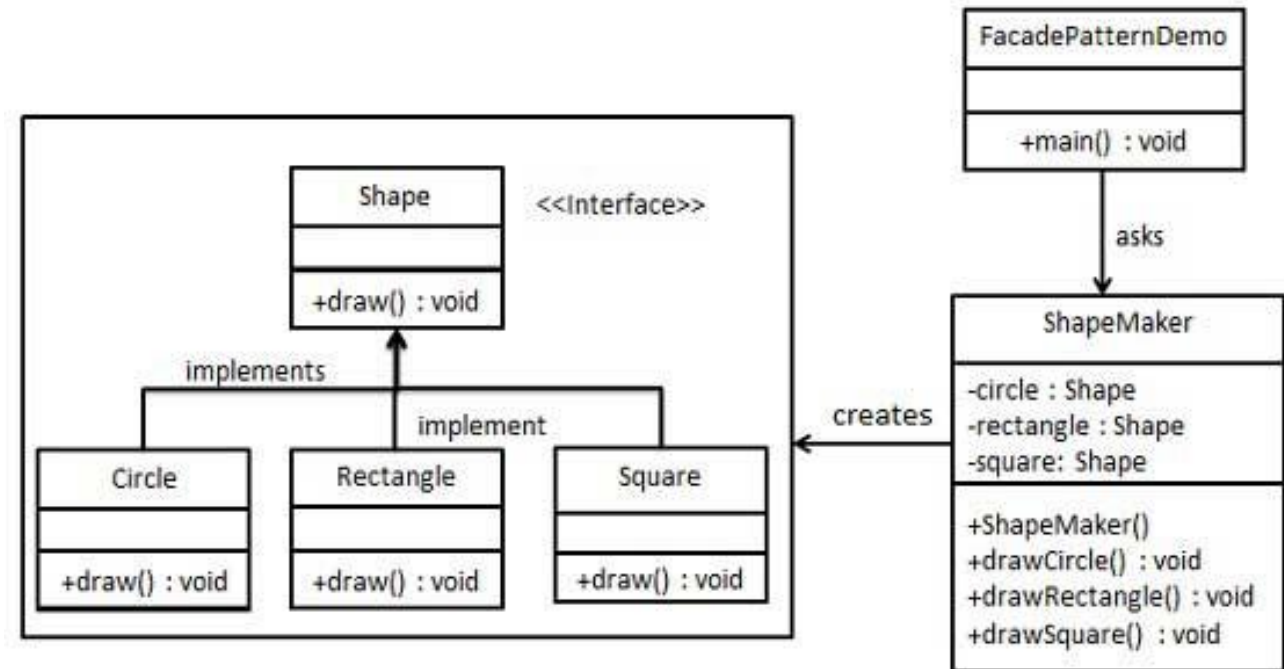
- **Class1** và **Class2** là các lớp đã có trong hệ thống.
- **Facade** là lớp sử dụng các phương thức của **Class1** và **Class2** để tạo ra một giao diện mới nên thường ít phức tạp hơn khi sử dụng riêng **Class1** và **Class2**.

I 7.5.3 TÌNH HUỐNG ÁP DỤNG MẪU THIẾT KẾ FACADE

- **Facade** làm cho một hệ thống phức tạp dễ sử dụng hơn bằng cách cung cấp một giao tiếp đơn giản mà không cần loại bỏ các lựa chọn phức tạp.
- **Facade** có thể được sử dụng cùng với mẫu **Abstract Factory** để cung cấp một giao diện hoạt động cho các hệ thống con độc lập. **Abstract Factory** cũng có thể được sử dụng như một sự thay thế cho **Facade** để ẩn các lớp nền đặc biệt.
- Tương tự như **Mediator** ở chỗ trừu tượng hóa chức năng của một lớp đã tồn tại. **Facade** chỉ đơn thuần là *trừu tượng giao diện cho một đối tượng hệ thống con để làm nó dễ sử dụng hơn nhưng không định nghĩa một chức năng mới* và lớp hệ thống con không hề biết gì về nó. Tuy nhiên, **Mediator** thường trừu tượng hóa chức năng trung tâm không thuộc về bất kỳ đối tượng cộng tác nào.

| 7.5.4 VÍ DỤ MẪU THIẾT KẾ FACADE

- Giả sử một hệ thống cũ đã có các thông tin về hình dạng **Circle**, **Square**, **Rectangle**. Ta xây dựng interface **Shape** và các lớp cụ thể cài đặt **Shape**.
- Một **facade ShapeMaker** được tạo ra để gửi lời gọi nhận được từ client đến các lớp cụ thể này.
- Lớp **FacadePatternDemo** là lớp client sẽ sử dụng lớp **ShapeMaker** để chạy ứng dụng.



| VÍ DỤ MẪU THIẾT KẾ FACADE

Bước 1: Tạo interface Shape

Shape.java

```
public interface Shape {  
    void draw();  
}
```

Bước 2: Tạo lớp cụ thể triển khai interface

Rectangle.java

```
public class Rectangle implements Shape {  
  
    @Override  
    public void draw() {  
        System.out.println("Rectangle::draw()");  
    }  
}
```

Square.java

```
public class Square implements Shape {  
  
    @Override  
    public void draw() {  
        System.out.println("Square::draw()");  
    }  
}
```

Circle.java

```
public class Circle implements Shape {  
  
    @Override  
    public void draw() {  
        System.out.println("Circle::draw()");  
    }  
}
```

I VÍ DỤ MẪU THIẾT KẾ FACADE

Bước 3: Tạo lớp facade

ShapeMaker.java

```
public class ShapeMaker {
    private Shape circle;
    private Shape rectangle;
    private Shape square;

    public ShapeMaker() {
        circle = new Circle();
        rectangle = new Rectangle();
        square = new Square();
    }

    public void drawCircle(){
        circle.draw();
    }
    public void drawRectangle(){
        rectangle.draw();
    }
    public void drawSquare(){
        square.draw();
    }
}
```

Bước 4: Sử dụng facade để vẽ các loại hình dạng khác nhau

FacadePatternDemo.java

```
public class FacadePatternDemo {
    public static void main(String[] args) {
        ShapeMaker shapeMaker = new ShapeMaker();

        shapeMaker.drawCircle();
        shapeMaker.drawRectangle();
        shapeMaker.drawSquare();
    }
}
```

Bước 5: Xem kết quả

```
Circle::draw()
Rectangle::draw()
Square::draw()
```

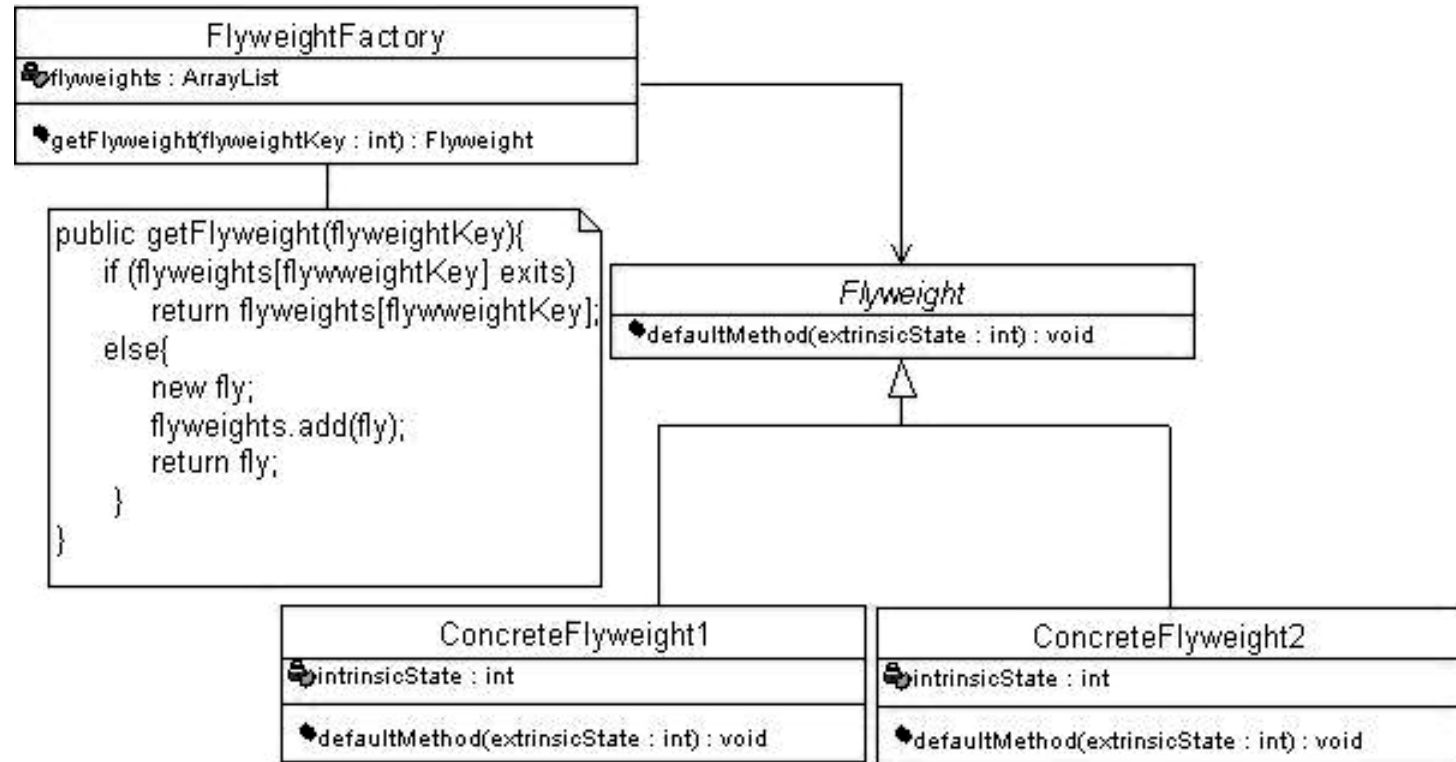

| 7.6 MẪU THIẾT KẾ FLYWEIGHT

7.6.1 Đặt vấn đề

- Mẫu **Flyweight** được sử dụng để thiết kế cách tạo đối tượng hiệu quả hơn. Một số ứng dụng có thể sử dụng các đối tượng xuyên suốt pha thiết kế, nhưng như vậy việc cài đặt không tốt sẽ gây nhiều khó khăn. Trong tình huống như vậy có thể dùng mẫu thiết kế **Flyweight** để giải quyết hiệu quả việc phối hợp hoạt động giữa một lượng lớn các đối tượng.
- Khi sử dụng mẫu này cần chú ý rằng các hiệu ứng của nó đòi hỏi phải biết rõ nó được sử dụng ở đâu và sử dụng như thế nào.
- Nên sử dụng mẫu này khi tất cả các điều sau đây thỏa mãn:
 - Ứng dụng sử dụng một số lượng lớn đối tượng.
 - Chi phí lưu trữ bởi số lượng các đối tượng là lớn.
 - Hầu hết trạng thái của các đối tượng có thể chịu tác động từ bên ngoài.
 - Ứng dụng không yêu cầu đối tượng đồng nhất vì các đối tượng **flyweight** có thể bị phân rã.

| 7.6.2 CẤU TRÚC MẪU FLYWEIGHT

- **FlyweightFactory**: tạo ra và quản lý các đối tượng **Flyweight**.
- **Flyweight**: là một giao tiếp interface định nghĩa các phương thức chuẩn
- **ConcreteFlyweightX**: là các lớp thực thi của **Flyweight**, các thể hiện của các lớp này sẽ được sử dụng tùy thuộc vào điều kiện bên ngoài.



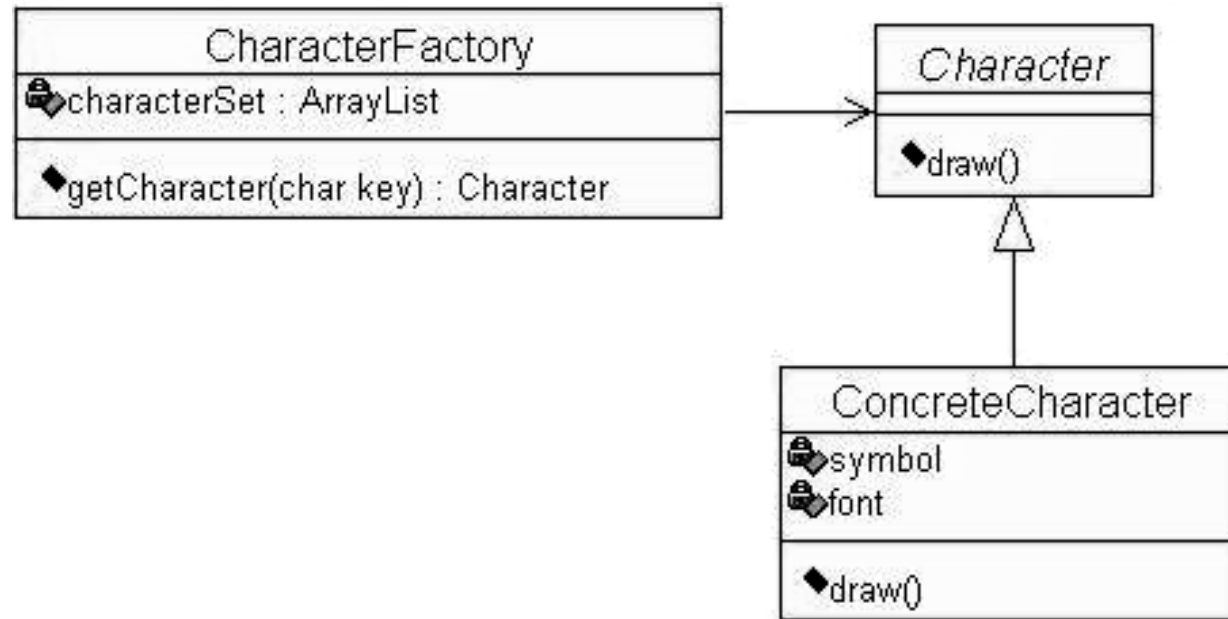
I 7.6.3 TÌNH HUỐNG ÁP DỤNG MẪU FLYWEIGHT

- Ứng dụng sử dụng nhiều đối tượng giống hoặc gần giống nhau. Với các đối tượng gần giống nhau, những phần không giống nhau có thể tách rời với các phần giống nhau để cho phép các phần giống nhau có thể chia sẻ.
- Nhóm các đối tượng gần giống nhau có thể được thay thế bởi một đối tượng chia sẻ mà các phần không giống nhau đã được loại bỏ.
- Nếu ứng dụng cần phân biệt các đối tượng gần giống nhau trong trạng thái gốc của chúng.
- Mẫu **Flyweight** thường kết hợp với mẫu **Composite** và thường tốt nhất là cài đặt các mẫu **State** và **Strategy** giống như là **flyweight**.

| 7.6.4 VÍ DỤ MẪU FLYWEIGHT

- Một ví dụ truyền thống của mẫu **flyweight** là các ký tự được lưu trong một bộ xử lý văn bản. Mỗi ký tự đại diện cho một đối tượng có dữ liệu là loại font, kích thước font, và các dữ liệu định dạng khác. Với một tài liệu lớn thì bộ xử lý văn bản sẽ gặp khó khăn khi xử lý với cấu trúc dữ liệu như thế này. Vì hầu hết dữ liệu dạng này là lặp lại nên phải có một cách để giảm việc lưu trữ này.
- Trong **Flyweight** mỗi đối tượng ký tự sẽ chứa một tham chiếu đến một đối tượng định dạng riêng rẽ mà chính đối tượng này sẽ chứa các thuộc tính cần thiết.

| 7.6.4 VÍ DỤ MẪU FLYWEIGHT



- Giao tiếp interface **Character** định nghĩa phương thức tạo một kí tự và Lớp kí tự **ConcreteCharacter** cài đặt giao tiếp **Character**.
- Sở dĩ ta định nghĩa **Character** và **ConcreteCharacter** riêng là vì trong cấu trúc sẽ có một phần nữa là phần không giống nhau giữa các đối tượng **Character** mà ở đây ta không bàn tới.

I VÍ DỤ MẪU FLYWEIGHT

```
public interface Character {
    public void draw();
}

public class ConcreteCharacter implements Character{
    private String symbol;
    private String font;
    public ConcreteCharacter(String s, String f){
        this.symbol = s;
        this.font = f;
    }
    public void draw() {
        System.out.println("Symbol " + this.symbol
            + " with font " + this.font );
    }
}

import java.util.*;
public class CharacterFactory {
    private Hashtable pool = new Hashtable<String, Character>();
    public int getNum() {
        return pool.size();
    }
    public Character get(String symbol, String fontFace) {
        Character c;
        String key = symbol + fontFace;
        if ((c = (Character)pool.get(key)) != null) {
            return c;
        } else {
            c = new ConcreteCharacter(symbol, fontFace);
            pool.put(key, c);
            return c;
        }
    }
}
```

```
//Lớp Test
import java.util.*;
public class Test {
    public static void main(String[] args) {
        CharacterFactory characterfactory = new CharacterFactory();
        ArrayList<Character> text = new ArrayList<Character>();
        text.add(0, characterfactory.get("a", "arial"));
        text.add(1, characterfactory.get("b", "time"));
        text.add(2, characterfactory.get("a", "arial"));
        text.add(0, characterfactory.get("c", "arial"));
        for (int i = 0; i < text.size(); i++){
            Character c = (Character)text.get(i);
            c.draw();
        }
    }
}
```

| 7.7 MẪU THIẾT KẾ PROXY

7.7.1 Đặt vấn đề

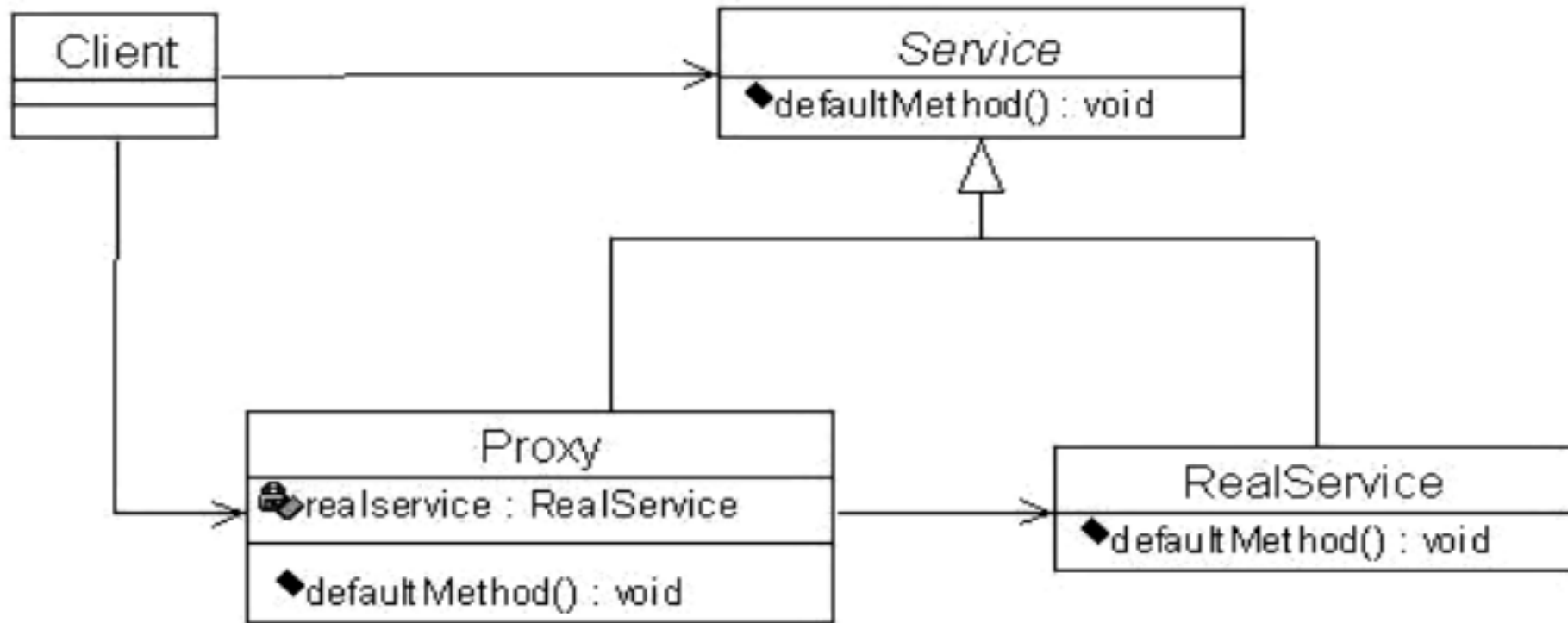
- Khi cần điều khiển truy nhập tới một đối tượng được thực hiện từ quá trình khởi tạo nó cho tới khi thực sự cần sử dụng nó. Trong trường hợp như vậy, ta nên dùng mẫu thiết kế **proxy**. Mẫu này có thể áp dụng vào những tình huống cần phải tham chiếu tới một đối tượng linh hoạt hơn, tinh tế hơn so với việc sử dụng con trỏ đơn giản.
- **Proxy** cung cấp một đại diện cho một đối tượng để điều khiển việc truy nhập nó. Có thể sử dụng **proxy** để đếm số tham chiếu tới đối tượng thực, do đó nó có thể tự động giải phóng khi không tham chiếu hay tải một đối tượng vào bộ nhớ khi nó được tham chiếu lần đầu tiên. Hoặc cần kiểm tra đối tượng thực nào đó có được khóa hay không trước khi nó bị truy nhập để đảm bảo không đối tượng nào khác có thể thay đổi nó.

| 7.7.2 MẪU THIẾT KẾ PROXY

Một số kiểu **proxy** thường được sử dụng:

- Một **remote proxy** từ xa cung cấp một biểu diễn cục bộ cho một đối tượng trong một không gian địa chỉ khác.
- Một **virtual proxy** ảo tạo ra một đối tượng có chi phí cao theo yêu cầu.
- Một **protection proxy** bảo vệ điều khiển việc truy nhập đối tượng gốc. Các **protection proxy** rất hữu ích khi các đối tượng có các quyền truy nhập khác nhau.
- Một **smart reference proxy** là sự thay thế cho một con trỏ rỗng cho phép thực hiện các chức năng thêm vào khi một đối tượng được truy nhập

| 7.7.2 CẤU TRÚC MẪU THIẾT KẾ PROXY

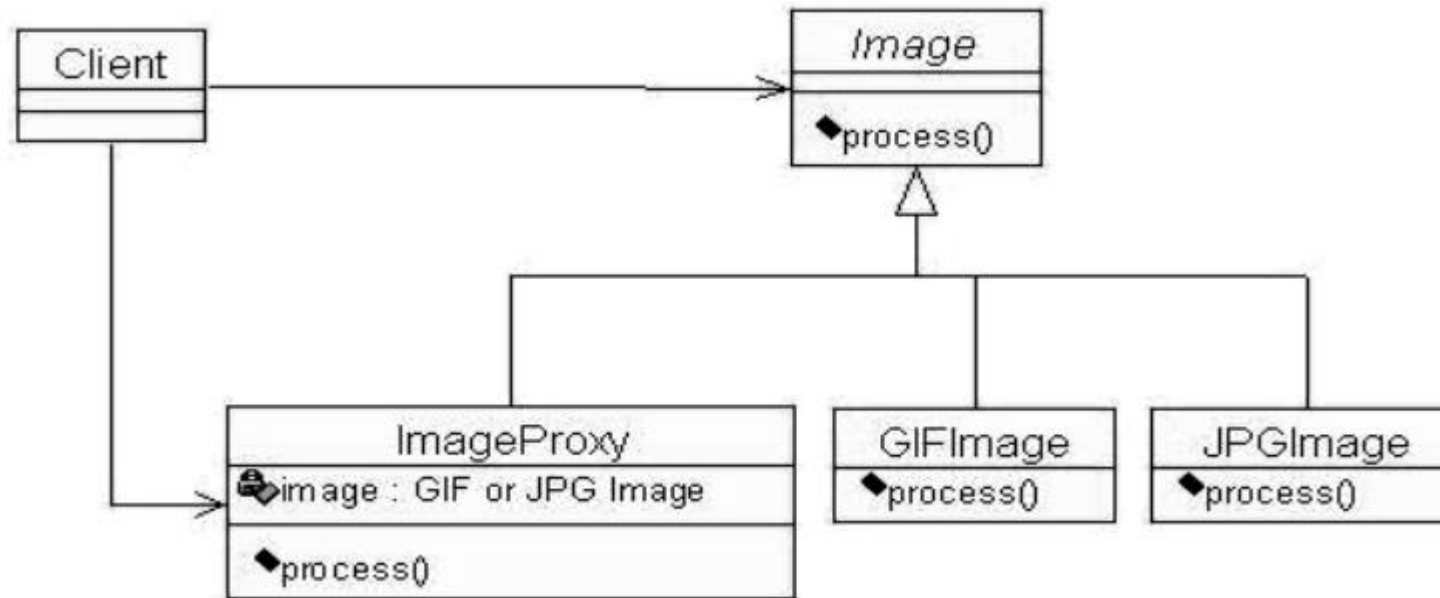


- **Service**: là giao tiếp định nghĩa các phương thức chuẩn cho một dịch vụ nào đó.
- **RealService**: là một cài đặt của giao tiếp **Service**, lớp này sẽ khai báo tường minh các phương thức của **Service**, lớp này xem như thực hiện tốt tất cả các yêu cầu từ **Service**.
- **Proxy**: kế thừa **Service** và sử dụng đối tượng của **RealService**.

| 7.7.3 TÌNH HUỐNG ÁP DỤNG MẪU PROXY

- Sử dụng mẫu **Proxy** khi cần một tham chiếu đến một đối tượng một cách phức tạp thay vì theo cách bình thường.
- **Proxy** truy nhập đối tượng từ xa (**Remote proxy**) – có thể sử dụng khi bạn cần một tham chiếu định vị cho một đối tượng trong không gian địa chỉ máy ảo Java (**JVM**).
- **Proxy ảo** (**Virtual proxy**) – lưu giữ các thông tin thêm vào về một dịch vụ thực vì vậy chúng có thể hoãn lại sự truy xuất vào dịch vụ này
- **Proxy bảo vệ** (**Protection proxy**) – xác thực quyền truy xuất vào một đối tượng thực.

| 7.7.4 VÍ DỤ MẪU THIẾT KẾ PROXY



- Lớp **Image** là một interface định nghĩa các phương thức xử lý ảnh và có lớp con là **GIFImage** và **JPGImage**.
- Trong trường hợp một lúc có thể đọc đến hàng trăm ảnh và còn muốn xử lý tùy vào một điều kiện nào đó, *nếu đặt điều kiện IF Image (sau đó sẽ tùy điều kiện này rồi xử lý riêng) thì không hợp lý; nếu đặt trong Client mỗi lần cần thay đổi IF lại sửa Client thì cũng không hợp lý* khi Client là một ứng dụng lớn -> Sử dụng **Proxy**.
- Khi sử dụng **Proxy**, lớp **ImageProxy** chỉ là lớp đại diện cho **Image**, kế thừa từ **Image** và sử dụng các lớp **GIFImage** hay **JPGImage**. Khi cần thay đổi ta chỉ cần thay đổi trên **Proxy** mà không cần tác động đến **Client** và **Image**.

| 7.7.4 VÍ DỤ MẪU THIẾT KẾ PROXY

```
public interface Image {  
    public void process();  
}  
  
public class JPGImage implements Image {  
    public void process() {  
        System.out.print("JPG Image");  
    }  
}  
  
public class GIFImage implements Image {  
    public void process() {  
        System.out.print("GIF Image");  
    }  
}  
  
public class ImageProxy implements Image {  
    private Image image;  
    public void process() {  
        //minh họa tạo đối tượng ảnh JPG  
        if (image == null)  
            image = new JPGImage();  
        image.process();  
    }  
}
```

| Bài tập thực hành

1. Cài đặt thực hành ví dụ các mẫu trong bài học, bao gồm **client** và hàm **main()** để chạy chương trình.
2. Giả sử có các lớp liên quan đến quản lý thông tin cá nhân gồm địa chỉ **Address**, số điện thoại **PhoneNum**, tên **Name**. Để quản lý các thông tin trên dựa vào tận dụng lại hệ thống cũ bằng cách xây một lớp người **Person** sử dụng lại các lớp ở trên. Sử dụng mẫu **facade** để xây dựng mã và thêm mã **client** với **main()** để thực thi chương trình.

